

cs210-assignment04

AI Disclosure:

I asked chatgpt how to use SDL2 library and how to add property names, cost and owner.

I also used chatgpt to generate property names.

For font in GUI, I asked ChatGPT how to use SDL_ttf.h library instead of using the "Tiny font".

Code

Woo_Matthew_Assignment4_part1_main.cpp

```
#include <iostream>
#include <iomanip>
#include <vector>
#include "Matthew_NumberList.h"
using namespace std;

int main() {
    vector<double> numbersToInsert = {
        77.75, 15.25, -4.25, 63.5, 18.25, -3.5
    };

    NumberList list;
    cout << fixed << setprecision(2);

    // Test adding and printing
    cout << "ADDING ITEMS\n";
    for (double number : numbersToInsert) {
        cout << "List after appending " << number << ": ";
        list.Append(number);
        list.Print(cout, ", ", "", "\n");
    }

    // Test searching
    cout << "\nSEARCHING\n";
    cout << "Search for 63.50: "
        << (list.Search(63.50) ? "Found" : "Not found") << '\n';
    cout << "Search for 100.00: "
        << (list.Search(100.00) ? "Found" : "Not found") << '\n';

    // Test removing from the front, middle, end, and a missing value
    cout << "\nREMOVING ITEMS\n";
```

```

vector<double> numbersToRemove = {
    77.75, 18.25, -3.5, 100.0
};

for (double toRemove : numbersToRemove) {
    cout << "Removing " << toRemove << ": ";

    if (list.Remove(toRemove)) {
        list.Print(cout, ", ", "", "\n");
    }
    else {
        cout << "value not found\n";
    }
}
cout << endl;
cout << "Searching for 63.5: ";
if (list.Search(63.5)) {
    cout << "Found" << endl;
}
else {
    cout << "Not found" << endl;
}

cout << "Searching for 100: ";
if (list.Search(100)) {
    cout << "Found" << endl;
}
else {
    cout << "Not found" << endl;
}

return 0;
}

```

Matthew_NumberList.h

```

#ifndef NUMBERLIST_H
#define NUMBERLIST_H
#include <iostream>
#include <string>
#include "NumberListNode.h"

// Base class for a double-linked list of NumberListNodes
class NumberList {

public:
    NumberListNode* head;
    NumberListNode* tail;
    NumberList() {
        head = nullptr;
        tail = nullptr;
    }

    virtual ~NumberList() {
    }
}

```

```

NumberListNode* GetHead() const {
    return head;
}

// Prints this list's contents in order from head to tail
void Print(std::ostream& output = std::cout, std::string separator = ", ",
    std::string prefix = "", std::string suffix = "") const {

    // Print prefix first
    output << prefix;

    // Start at the list's head
    NumberListNode* node = head;

    // First node's data is printed without accompanying separator
    if (node) {
        output << node->GetData();
        node = node->GetNext();
    }
    else {
        // Special case for empty list
        output << "(empty)";
    }

    // Remaining nodes are printed with separator before
    while (node) {
        output << separator << node->GetData();
        node = node->GetNext();
    }

    // Suffix is printed last
    output << suffix;
}

void Append(double number) {
    NumberListNode* newNode = new NumberListNode(number, nullptr, nullptr);

    if (head == nullptr) {
        head = newNode;
        tail = newNode;
    }
    else {
        tail->SetNext(newNode);
        tail = newNode;
    }
}

void Prepend(double number){
    NumberListNode* newNode = new NumberListNode(number, nullptr,
nullptr);

    if (head == nullptr){
        head = newNode;
        tail = newNode;
    }
}

```

```

        else{
            newNode->SetNext(head);
            head = newNode;
        }
    }

    bool Search(double number) {
        NumberListNode* current = head;

        while (current != nullptr) {
            if (current->GetData() == number) {
                return true;
            }

            current = current->GetNext();
        }
        return false;
    }

    bool Remove(double number) {
        NumberListNode* current = head;
        NumberListNode* previous = nullptr;

        while (current != nullptr && current->GetData() != number) {
            previous = current;
            current = current->GetNext();
        }

        // Number was not found
        if (current == nullptr) {
            return false;
        }

        // Removing the head
        if (current == head) {
            head = head->GetNext();
            // If only one node, then point tail
            if (head == nullptr) {
                tail = nullptr;
            }
        }

        // Removing somewhere after the head
        else {
            previous->SetNext(current->GetNext());

            if (current == tail) {
                tail = previous;
            }
        }

        delete current;
        return true;
    }
};

```

```
#endif
```

```
NumberListNode.h
```

```
#ifndef NUMBERLISTNODE_H
```

```
#define NUMBERLISTNODE_H
```

```
class NumberListNode {
```

```
protected:
```

```
    double data;
```

```
    NumberListNode* next;
```

```
    NumberListNode* previous;
```

```
public:
```

```
    // Constructs this node with the specified numerical data value. The next  
    // and previous pointers are each assigned nullptr.
```

```
    NumberListNode(double initialData) {
```

```
        data = initialData;
```

```
        next = nullptr;
```

```
        previous = nullptr;
```

```
    }
```

```
    // Constructs this node with the specified numerical data value, next  
    // pointer, and previous pointer.
```

```
    NumberListNode(double initialData, NumberListNode* nextNode,
```

```
        NumberListNode* previousNode) {
```

```
        data = initialData;
```

```
        next = nextNode;
```

```
        previous = previousNode;
```

```
    }
```

```
    virtual ~NumberListNode() {
```

```
    }
```

```
    // Returns this node's data.
```

```
    virtual double GetData() {
```

```
        return data;
```

```
    }
```

```
    // Sets this node's data.
```

```
    virtual void SetData(double newData) {
```

```
        data = newData;
```

```
    }
```

```
    // Gets this node's next pointer.
```

```
    virtual NumberListNode* GetNext() {
```

```
        return next;
```

```
    }
```

```
    // Sets this node's next pointer.
```

```
    virtual void SetNext(NumberListNode* newNext) {
```

```
        next = newNext;
```

```
    }
```

```
    // Gets this node's previous pointer.
```

```

    virtual NumberListNode* GetPrevious() {
        return previous;
    }

    // Sets this node's previous pointer.
    virtual void SetPrevious(NumberListNode* newPrevious) {
        previous = newPrevious;
    }
};

```

#endif

Woo_Matthew_Assignment4_monopoly

```

// Based on Professor Dominic's version monopoly
// JDoodle: Libraries = SDL2; enable GUI and Interactive.

/**
AI Disclosure:
I asked chatgpt how to use SDL2 library and how to add property names, cost
and owner.
I also used chatgpt to generate property names.
For font in GUI, I asked ChatGPT how to use SDL_ttf.h library instead of
using the "Tiny font".
**/
#include <SDL2/SDL.h>
#include <SDL2/SDL_ttf.h> // font library
#include <cmath>
#include <cstdlib>
#include <ctime>
#include <iostream>
#include <string>
using namespace std;

struct Node {
    int value;
    string name;
    int cost;
    string owner;
    Node* next;
};

/**
// Tiny font: no extra font library or font files needed.
void text(SDL_Renderer* r, string words, int x, int y, int size) {
    string letters = "123456789ROLP";
    string font[] = {
        "010110010010111", "111001111100111", "111001111001111",
        "1011011111001001", "111100111001111", "111100111101111",
        "111001001001001", "111101111101111", "111101111001111",
        "110101110101101", "111101101101111", "100100100100111",
        "110101110100100"
    };
    for (char c : words) {
        auto letter = letters.find(c);
        if (letter != string::npos)
            for (int i = 0; i < 15; i++)

```

```

        if (font[letter][i] == '1') {
            SDL_Rect pixel = {x + i % 3 * size, y + i / 3 * size,
size, size};
            SDL_RenderFillRect(r, &pixel);
        }
        x += 4 * size;
    }
}
**/

// I asked ChatGPT for how to use the SDL_ttf.h library in this part
// Draw normal text using SDL_ttf
void drawText(SDL_Renderer* renderer,
              TTF_Font* font,
              string message,
              int x,
              int y,
              SDL_Color color) {

    // Convert the text into an SDL surface
    SDL_Surface* surface =
        TTF_RenderText_Blended(font, message.c_str(), color);

    // Check if the text was created successfully
    if (surface == nullptr) {
        cerr << "Text error: " << TTF_GetError() << endl;
        return;
    }

    // Convert the surface into a texture so SDL can draw it
    SDL_Texture* texture =
        SDL_CreateTextureFromSurface(renderer, surface);

    // Check if the texture was created successfully
    if (texture == nullptr) {
        SDL_FreeSurface(surface);
        return;
    }

    // Set where the text will appear
    SDL_Rect textRect = {
        x,
        y,
        surface->w,
        surface->h
    };

    // Draw the text
    SDL_RenderCopy(renderer, texture, nullptr, &textRect);

    // Free memory after drawing
    SDL_FreeSurface(surface);
    SDL_DestroyTexture(texture);
}

```

```

// Put each space around a circle, clockwise from the top.
SDL_Point position(int index) {
    double angle = index * 2 * 3.14159 / 30 - 3.14159 / 2;
    return {360 + int(285 * cos(angle)), 360 + int(285 * sin(angle))};
}

int main() {
    srand(static_cast<unsigned>(time(nullptr)));
    if (SDL_Init(SDL_INIT_VIDEO) != 0) {
        cerr << SDL_GetError() << '\n';
        return 1;
    }
    // Start the SDL_ttf text library
    if (TTF_Init() == -1) {
        cerr << "TTF Error: " << TTF_GetError() << endl;
        return 1;
    }
    SDL_Window* window = SDL_CreateWindow(
        "P1 (blue) starts | Click the die or press SPACE",
        SDL_WINDOWPOS_CENTERED, SDL_WINDOWPOS_CENTERED, 1020, 720, 0); // I
made the window wider so it can display the turns info at the right side
    SDL_Renderer* r = window ? SDL_CreateRenderer(window, -1,
SDL_RENDERER_SOFTWARE) : nullptr;
    if (!r) {
        cerr << SDL_GetError() << '\n';
        if (window) SDL_DestroyWindow(window);
        SDL_Quit();
        return 1;
    }
    // Location of the font file
    const char* fontPath =
        "/System/Library/Fonts/Helvetica.ttc"; // For MacOS
        // "/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf"; //For linux

    // Load three different font sizes
    TTF_Font* smallFont = TTF_OpenFont(fontPath, 16);
    TTF_Font* mediumFont = TTF_OpenFont(fontPath, 24);
    TTF_Font* largeFont = TTF_OpenFont(fontPath, 48);

    // Make sure the fonts loaded correctly
    if (!smallFont || !mediumFont || !largeFont) {
        cerr << "Could not load font: "
            << TTF_GetError() << endl;
        return 1;
    }
    // Text colors used in the GUI
    SDL_Color white = {255, 255, 255, 255};
    SDL_Color dark = {25, 45, 40, 255};
    SDL_Color blue = {35, 125, 245, 255};
    SDL_Color red = {235, 65, 75, 255};
    // I used ChatGPT to generate the property name
    string propertyNames[30] = {
        "Mediterranean Avenue",
        "Baltic Avenue",
        "Oriental Avenue",

```

```

    "Vermont Avenue",
    "Connecticut Avenue",
    "St Charles Place",
    "States Avenue",
    "Virginia Avenue",
    "St James Place",
    "Tennessee Avenue",
    "New York Avenue",
    "Kentucky Avenue",
    "Indiana Avenue",
    "Illinois Avenue",
    "Atlantic Avenue",
    "Ventnor Avenue",
    "Marvin Gardens",
    "Pacific Avenue",
    "North Carolina Avenue",
    "Pennsylvania Avenue",
    "Park Place",
    "Boardwalk",
    "Reading Railroad",
    "Pennsylvania Railroad",
    "B and O Railroad",
    "Short Line",
    "Electric Company",
    "Water Works",
    "Harbor Avenue",
    "Sunset Avenue"
};

int propertyCosts[30] = {
    60, 60, 100, 100, 120,
    140, 140, 160, 180, 180,
    200, 220, 220, 240, 260,
    260, 280, 300, 300, 320,
    350, 400, 200, 200, 200,
    200, 150, 150, 180, 240
};

// 30 nodes, each holding a random value from 1 to 9.
Node* first = new Node{
    rand() % 9 + 1,
    propertyNames[0],
    propertyCosts[0],
    "Unowned",
    nullptr
};

Node* last = first;

for (int i = 1; i < 30; i++) {
    last->next = new Node{
        rand() % 9 + 1,
        propertyNames[i],
        propertyCosts[i],
        "Unowned",

```

```

        nullptr
    };

    last = last->next;
}

last->next = first; // Last space links back to the first.
Node* player[2] = {first, first};
// Remember the property that was most recently landed on
Node* lastProperty = first;
int turn = 0; // 0 = blue, 1 = red
int money[2] = {1000, 1000}; // Initial Budget for buying properties
int turnsPlayed = 0; // Count turns

int dice = 1;
SDL_Rect rollButton = {310, 285, 100, 100};
bool running = true;
SDL_Event event;
while (running) {
    while (SDL_PollEvent(&event)) {
        if (event.type == SDL_QUIT) running = false;
        bool roll = event.type == SDL_KEYDOWN && !event.key.repeat &&
            event.key.keysym.sym == SDLK_SPACE;
        if (event.type == SDL_MOUSEBUTTONDOWN && event.button.button ==
SDL_BUTTON_LEFT) {
            SDL_Point mouse = {event.button.x, event.button.y};
            roll = SDL_PointInRect(&mouse, &rollButton);
        }
        if (roll) {
            dice = rand() % 6 + 1; //dice: 1-6
            for (int step = 0; step < dice; step++)
                player[turn] = player[turn]->next;

            // Save the property the player landed on
            lastProperty = player[turn];

            turnsPlayed++; //count turns

            cout << "Turn " << turnsPlayed << endl;
            cout << "P" << turn + 1 << " rolled " << dice << endl;
            cout << "Landed on " << player[turn]->name << endl;
            cout << "Cost: $" << player[turn]->cost << endl;
            cout << "Owner: " << player[turn]->owner << endl;

            // Buy the property if it is unowned
            if (player[turn]->owner == "Unowned") {

                if (money[turn] >= player[turn]->cost) {
                    money[turn] -= player[turn]->cost;

                    player[turn]->owner =
                        "P" + to_string(turn + 1);

                    cout << "P" << turn + 1
                        << " bought "

```

```

        << player[turn]->name
        << " for $"
        << player[turn]->cost
        << endl;

        cout << "Money left: $"
            << money[turn]
            << endl;
    }
    else {
        cout << "P" << turn + 1
            << " does not have enough money."
            << endl;
    }
}

// Property already has an owner
else {
    cout << player[turn]->name
        << " is already owned by "
        << player[turn]->owner
        << ". Cannot buy this property."
        << endl;
}

cout << endl;

// Show the current property information in GUI title
string title =
    "P" + to_string(turn + 1) +
    " rolled " + to_string(dice) +
    " | " + player[turn]->name +
    " | Cost $" + to_string(player[turn]->cost) +
    " | Owner: " + player[turn]->owner;

// Print results after 10 turns
if (turnsPlayed == 10) {
    cout << "==== RESULTS AFTER 10 TURNS =====" << endl;

    cout << "P1 Money: $" << money[0] << endl;
    cout << "P2 Money: $" << money[1] << endl;

    cout << endl;
    cout << "Property Results:" << endl;

    Node* result = first;

    for (int i = 0; i < 30; i++) {
        cout << result->name
            << " | Cost: $" << result->cost
            << " | Owner: " << result->owner
            << endl;

        result = result->next;
    }
}

```

```

        cout << "===== " << endl;
    }

    turn = 1 - turn; // Switch players.
    title += " | P" + to_string(turn + 1) + " turn | Click die or
SPACE";
    SDL_SetWindowTitle(window, title.c_str());
}

SDL_SetRenderDrawColor(r, 22, 65, 56, 255);
SDL_RenderClear(r);
// Create a panel on the right side of the window
SDL_Rect infoPanel = {
    720,    // x
    0,      // y
    300,    // width
    720     // height
};

// Give the information panel a light background
SDL_SetRenderDrawColor(r, 235, 235, 235, 255);
SDL_RenderFillRect(r, &infoPanel);

// Title of the information panel
drawText(
    r,
    mediumFont,
    "Game Info",
    740,
    30,
    dark
);

// Show how many turns have been played
drawText(
    r,
    smallFont,
    "Turn: " + to_string(turnsPlayed),
    740,
    80,
    dark
);

// Property section title
drawText(
    r,
    smallFont,
    "Property:",
    740,
    140,
    dark
);

```

```

// Show the property name
drawText(
    r,
    smallFont,
    lastProperty->name,
    740,
    170,
    dark
);

// Show the property cost
drawText(
    r,
    smallFont,
    "Cost: $" + to_string(lastProperty->cost),
    740,
    210,
    dark
);

// Show the property owner
drawText(
    r,
    smallFont,
    "Owner: " + lastProperty->owner,
    740,
    240,
    dark
);

// Show both players' current money in the top-left
drawText(
    r,
    smallFont,
    "P1 Money: $" + to_string(money[0]),
    15,
    15,
    white
);

drawText(
    r,
    smallFont,
    "P2 Money: $" + to_string(money[1]),
    15,
    40,
    white
);

// Draw the circular path behind the spaces.
Node* current = first;
SDL_SetRenderDrawColor(r, 110, 150, 130, 255);
for (int i = 0; i < 30; i++) {
    SDL_Point a = position(i), b = position((i + 1) % 30);

```

```

        SDL_RenderDrawLine(r, a.x, a.y, b.x, b.y);
    }

    // Walk the linked list to draw the board and both players.
    int index = 0;
    do {
        SDL_Point p = position(index++);
        SDL_Rect tile = {p.x - 20, p.y - 20, 40, 40};
        SDL_SetRenderDrawColor(r, 239, 237, 219, 255);
        SDL_RenderFillRect(r, &tile);
        SDL_SetRenderDrawColor(r, 25, 45, 40, 255);
        // text(r, to_string(current->value), p.x - 6, p.y - 15, 4);
        // Draw the number inside each property square
        drawText(
            r,
            smallFont,
            to_string(current->value),
            p.x - 6,
            p.y - 15,
            dark
        );
        for (int j = 0; j < 2; j++) {
            if (current != player[j]) continue;
            SDL_Rect token = {p.x - 14 + j * 18, p.y + 8, 10, 10};
            if (j == 0) SDL_SetRenderDrawColor(r, 35, 125, 245, 255);
            else SDL_SetRenderDrawColor(r, 235, 65, 75, 255);
            SDL_RenderFillRect(r, &token);
        }
        current = current->next;
    } while (current != first);

    // Center button shows the latest roll.
    SDL_SetRenderDrawColor(r, 255, 255, 255, 255);
    SDL_RenderFillRect(r, &rollButton);
    // text(r, "ROLL", 315, 407, 6);
    // Draw ROLL text
    drawText(
        r,
        mediumFont,
        "ROLL",
        315,
        407,
        dark
    );
    SDL_SetRenderDrawColor(r, 25, 45, 40, 255);
    // text(r, to_string(dice), 342, 305, 12);
    // Draw latest dice number
    drawText(
        r,
        largeFont,
        to_string(dice),
        335,
        300,
        dark
    );
};

```

```

SDL_SetRenderDrawColor(r, 35, 125, 245, 255);
//text(r, "P1", 280, 475, 6);
// Player 1 label
drawText(
    r,
    mediumFont,
    "P1",
    280,
    475,
    blue
);
SDL_SetRenderDrawColor(r, 235, 65, 75, 255);
//text(r, "P2", 398, 475, 6);
// Player 2 label
drawText(
    r,
    mediumFont,
    "P2",
    398,
    475,
    red
);
/**
if (turn == 0) SDL_SetRenderDrawColor(r, 35, 125, 245, 255);
else SDL_SetRenderDrawColor(r, 235, 65, 75, 255);
text(r, "P" + to_string(turn + 1), 332, 210, 8);
**/

// Show whose turn it is
if (turn == 0) {
    drawText(
        r,
        mediumFont,
        "P" + to_string(turn + 1),
        332,
        210,
        blue
    );
}
else {
    drawText(
        r,
        mediumFont,
        "P" + to_string(turn + 1),
        332,
        210,
        red
    );
}

SDL_RenderPresent(r);
SDL_Delay(16);
}

// Delete each node once. Stop after 30, since the list is circular.

```

```

    for (int i = 0; i < 30; i++) {
        Node* next = first->next;
        delete first;
        first = next;
    }
    // Close all loaded fonts
    TTF_CloseFont(smallFont);
    TTF_CloseFont(mediumFont);
    TTF_CloseFont(largeFont);

    // Destroy SDL GUI objects
    SDL_DestroyRenderer(r);
    SDL_DestroyWindow(window);

    // Shut down SDL_ttf
    TTF_Quit();

    // Shut down SDL
    SDL_Quit();
}

```

Output

Part 1:

ADDING ITEMS

```

List after appending 77.75: 77.75
List after appending 15.25: 77.75, 15.25
List after appending -4.25: 77.75, 15.25, -4.25
List after appending 63.50: 77.75, 15.25, -4.25, 63.50
List after appending 18.25: 77.75, 15.25, -4.25, 63.50, 18.25
List after appending -3.50: 77.75, 15.25, -4.25, 63.50, 18.25, -3.50

```

SEARCHING

```

Search for 63.50: Found
Search for 100.00: Not found

```

REMOVING ITEMS

```

Removing 77.75: 15.25, -4.25, 63.50, 18.25, -3.50
Removing 18.25: 15.25, -4.25, 63.50, -3.50
Removing -3.50: 15.25, -4.25, 63.50
Removing 100.00: value not found

```

```

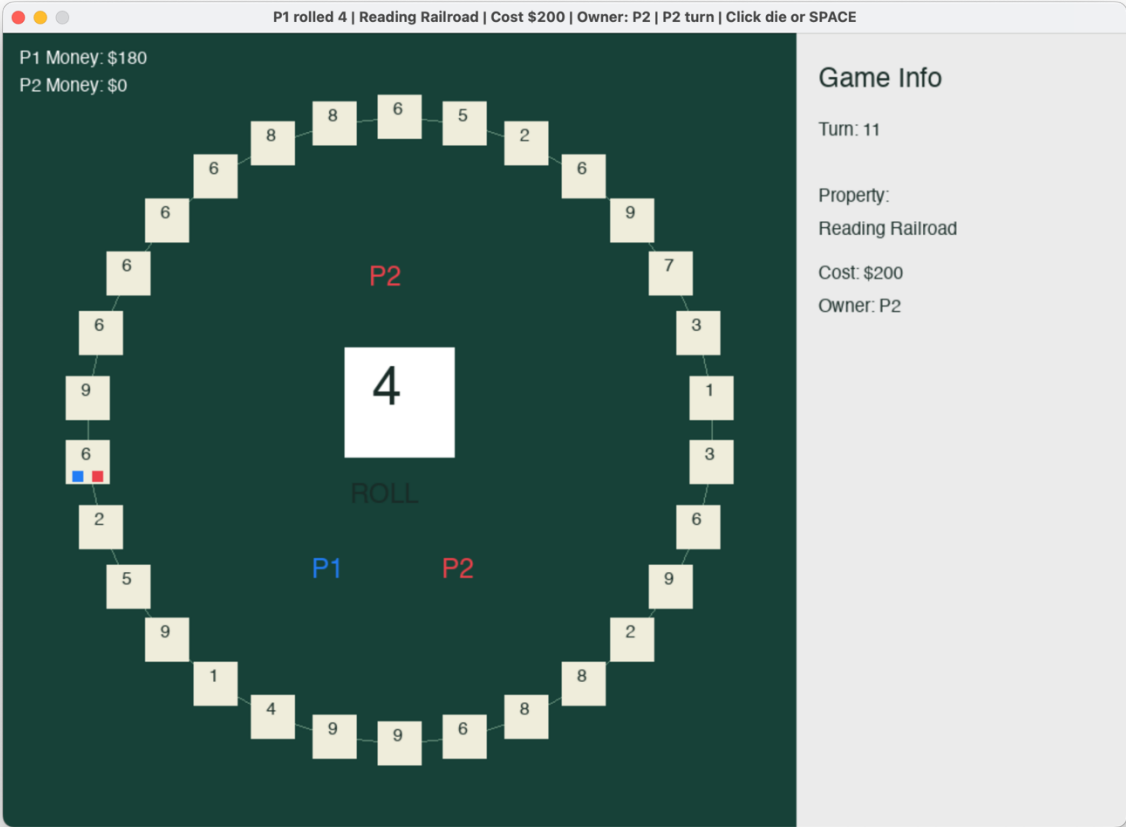
Searching for 63.5: Found
Searching for 100: Not found

```

Part 2 Monopoly: Initial Game GUI:



Turn 11 GUI:



```
Turn 1
P1 rolled 4
Landed on Connecticut Avenue
Cost: $120
Owner: Unowned
P1 bought Connecticut Avenue for $120
Money left: $880
```

```
Turn 3
P1 rolled 1
Landed on St Charles Place
Cost: $140
Owner: P2
St Charles Place is already owned by P2. Cannot buy this property.
```

Turn 4
P2 rolled 1
Landed on States Avenue
Cost: \$140
Owner: Unowned
P2 bought States Avenue for \$140
Money left: \$720

Turn 5
P1 rolled 2
Landed on Virginia Avenue
Cost: \$160
Owner: Unowned
P1 bought Virginia Avenue for \$160
Money left: \$720

Turn 6
P2 rolled 6
Landed on Indiana Avenue
Cost: \$220
Owner: Unowned
P2 bought Indiana Avenue for \$220
Money left: \$500

Turn 7
P1 rolled 6
Landed on Illinois Avenue
Cost: \$240
Owner: Unowned
P1 bought Illinois Avenue for \$240
Money left: \$480

Turn 8
P2 rolled 5
Landed on Pacific Avenue
Cost: \$300
Owner: Unowned
P2 bought Pacific Avenue for \$300
Money left: \$200

Turn 9
P1 rolled 5
Landed on North Carolina Avenue
Cost: \$300
Owner: Unowned
P1 bought North Carolina Avenue for \$300
Money left: \$180

Turn 10
P2 rolled 5
Landed on Reading Railroad
Cost: \$200
Owner: Unowned
P2 bought Reading Railroad for \$200

Money left: \$0

===== RESULTS AFTER 10 TURNS =====

P1 Money: \$180

P2 Money: \$0

Property Results:

Mediterranean Avenue | Cost: \$60 | Owner: Unowned

Baltic Avenue | Cost: \$60 | Owner: Unowned

Oriental Avenue | Cost: \$100 | Owner: Unowned

Vermont Avenue | Cost: \$100 | Owner: Unowned

Connecticut Avenue | Cost: \$120 | Owner: P1

St Charles Place | Cost: \$140 | Owner: P2

States Avenue | Cost: \$140 | Owner: P2

Virginia Avenue | Cost: \$160 | Owner: P1

St James Place | Cost: \$180 | Owner: Unowned

Tennessee Avenue | Cost: \$180 | Owner: Unowned

New York Avenue | Cost: \$200 | Owner: Unowned

Kentucky Avenue | Cost: \$220 | Owner: Unowned

Indiana Avenue | Cost: \$220 | Owner: P2

Illinois Avenue | Cost: \$240 | Owner: P1

Atlantic Avenue | Cost: \$260 | Owner: Unowned

Ventnor Avenue | Cost: \$260 | Owner: Unowned

Marvin Gardens | Cost: \$280 | Owner: Unowned

Pacific Avenue | Cost: \$300 | Owner: P2

North Carolina Avenue | Cost: \$300 | Owner: P1

Pennsylvania Avenue | Cost: \$320 | Owner: Unowned

Park Place | Cost: \$350 | Owner: Unowned

Boardwalk | Cost: \$400 | Owner: Unowned

Reading Railroad | Cost: \$200 | Owner: P2

Pennsylvania Railroad | Cost: \$200 | Owner: Unowned

B and O Railroad | Cost: \$200 | Owner: Unowned

Short Line | Cost: \$200 | Owner: Unowned

Electric Company | Cost: \$150 | Owner: Unowned

Water Works | Cost: \$150 | Owner: Unowned

Harbor Avenue | Cost: \$180 | Owner: Unowned

Sunset Avenue | Cost: \$240 | Owner: Unowned

=====

Turn 11

P1 rolled 4

Landed on Reading Railroad

Cost: \$200

Owner: P2

Reading Railroad is already owned by P2. Cannot buy this property.

2026-10-04 23:23:06.524 Woo_Matthew_Assignment4_monopoly[63433:2936971] TSM
AdjustCapsLockLEDForKeyTransitionHandling - _ISetPhysicalKeyboardCapsLockLED
Inhibit

Turn 12

P2 rolled 2

Landed on B and O Railroad

Cost: \$200

Owner: Unowned

P2 does not have enough money.

Turn 13
P1 rolled 6
Landed on Harbor Avenue
Cost: \$180
Owner: Unowned
P1 bought Harbor Avenue for \$180
Money left: \$0
It will print the results after turn 10, but will not end.

Explanation

Operation	Time Complexity
Adding	$O(1)$
Searching	$O(n)$
Removing	$O(n)$
Moving	$O(k)$
Traversing	$O(n)$

Adding is $O(1)$, because adding usually means adding the node to the tail, so it is $O(1)$.

Searching in the worst case might need to look for every element to search for the nodes, so it should be $O(n)$.

Removing is similar to searching, it also need to look for every element to search for the nodes to remove elements. So it is $O(n)$.

Moving is $O(k)$ because moving on the monopoly game board usually move k times, because the next pointer k times, and k is the dice roll.

Traversing is $O(n)$ because it is only visited once.